

LAB REPORT / 15 SEPTEMBER 2026

Clickjacking

The Button You Didn't Click

Name: Imad**Scope: Sections 1–6**

1. Objective and local setup

The objective was to build a local Student Portal and a *Catch Me!* game, embed the portal inside the game, and make a click intended for the game activate the portal's Login button. The experiment uses only the local pages supplied with this report.

`portal.html` contains the Student Portal, its name and password fields, and the Login button. Its `login()` function displays *You are logged in!* in the page. This is a simulated login: the function does not validate credentials or create an authenticated session.

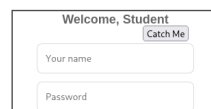
`game.html` contains the game button, the score, the movement logic, and a 600×600 embedded portal. The HyperText Markup Language (HTML) elements, Cascading Style Sheets (CSS), and JavaScript needed in Section 1 are used directly in these two pages.

Sections 2 and 4: portal creation and embedding

The initial embedded view clipped the portal. Enlarging the iframe made the full form and Login button visible, providing a usable reference for alignment.

Catch Me!

Score: 0

A small, cropped view of the Student Portal form. It shows the text "Welcome, Student" and a "Catch Me!" button in the top right corner. Below these are the "Your name" and "Password" input fields. The "Login" button is not visible as it is cut off by the bottom of the frame.

Catch Me!

Score: 0

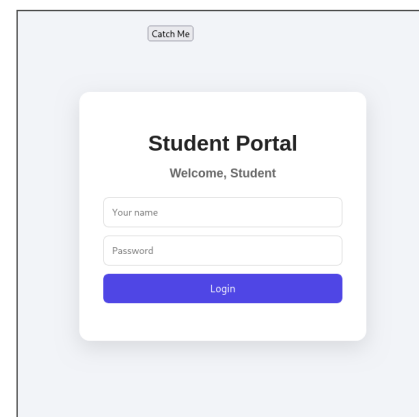
A larger view of the Student Portal form, showing the complete interface. It includes the "Welcome, Student" text, the "Catch Me!" button, the "Your name" and "Password" input fields, and a prominent blue "Login" button at the bottom. The entire form is centered within a light blue rectangular container.

Figure 1: Initial iframe: only part of the portal is visible.

Figure 2: Enlarged iframe: the complete portal and Login control are visible.

2. Game behavior and portal transparency

Section 3: the decoy game

The game begins at a displayed score of 0. Each call to `catchMe()` displays the current counter, increments it, and changes the button position. Before the threshold is reached, two random values set its vertical and horizontal positions:

```
x = Math.random() * 500;  
y = Math.random() * 400;  
button.style.top = `${x}px`;  
button.style.left = `${y}px`;
```

In this implementation, `x` is used for the vertical offset and `y` for the horizontal offset. The changing positions create the game interaction that precedes the final overlap.

Section 5: reducing visibility

The `iframe`'s opacity was reduced while retaining the embedded portal in the page. The screenshots below record two progressively fainter views. The final saved stylesheet uses `opacity: 0.5`, keeping the alignment visible for inspection.

Catch Me!

Score: 0

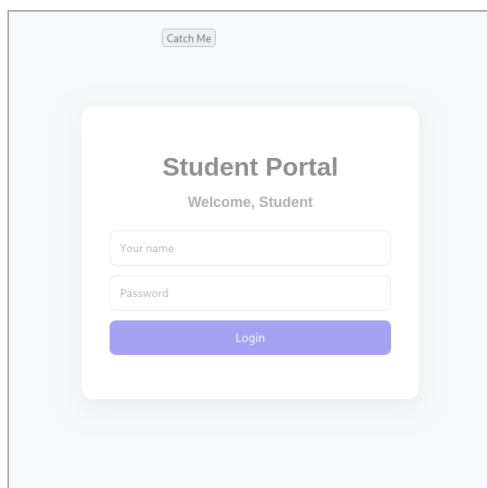


Figure 3: Partially transparent portal during the visibility experiment.

Catch Me!

Score: 0

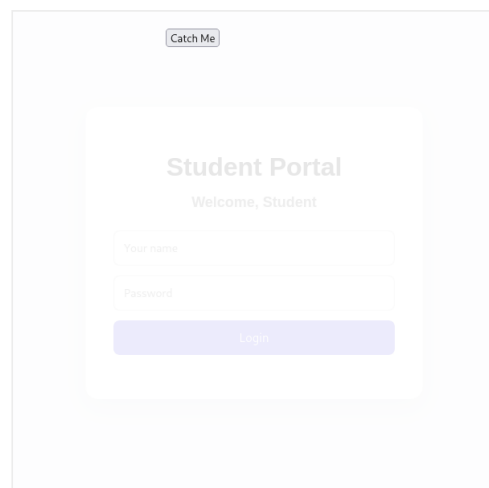


Figure 4: Further reduction in visibility: the portal becomes much fainter.

Observation

Transparency changes the appearance of the embedded page; it does not itself transfer a click to Login. The final interaction also requires overlapping controls and the `iframe` above the game button. The supplied screenshots document reduced opacity; the successful sequence on page 4 uses a partially visible portal.

3. Section 6: correcting the clickjacking moment

Problem encountered

An intermediate attempt placed the game button over the Login region, but the game still received clicks. In the earlier code, `button.syle.left` and `framei.syle.zIndex` misspelled `style`. At the threshold, the first typo caused this JavaScript error:

```
Cannot set properties of undefined (setting 'left')
```

Execution stopped before lowering the game button from its initial `z-index: 3`. The iframe remained at `z-index: 2`, underneath the game button. Visual overlap alone therefore did not produce the intended interaction.

Correction in the submitted code

Both spellings were corrected. The threshold branch now sets a fixed position and places the game button below the iframe:

```
if (scored > barrier) {
  button.style.top = "510px";
  button.style.left = "260px";
  button.style.zIndex = "1";
  framei.style.zIndex = "2";
}
```

Catch Me!

Score: 5

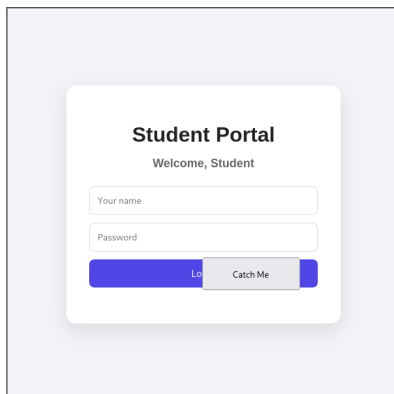


Figure 5: Intermediate attempt: overlap is visible, but the score has advanced to 5.

Catch Me!

Score: 0

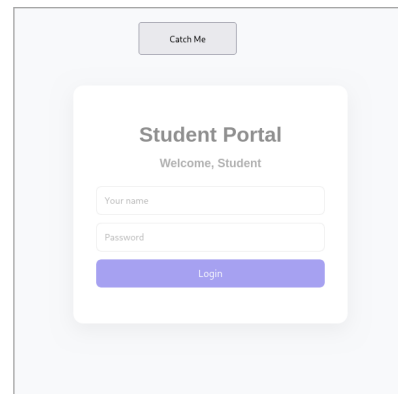


Figure 6: Corrected run: the game starts at score 0 with a partially visible portal.

The score counter starts at 1 internally, while the displayed score starts at 0. On the fourth game click, the page displays 4, increments the counter to 5, and enters the branch because $5 > 4$.

4. Section 6: observed result

Catch Me!

Score: 4

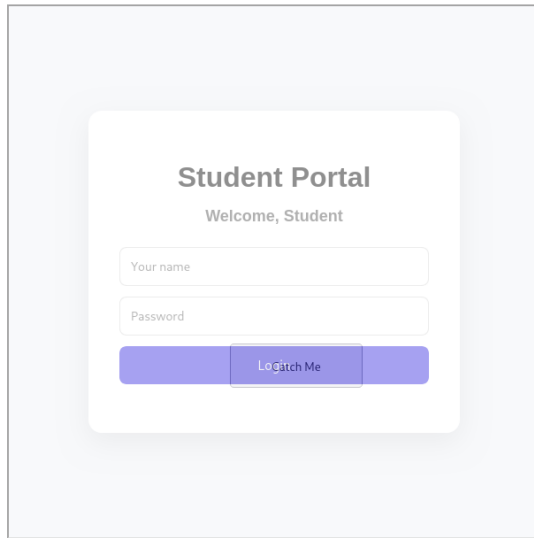


Figure 7: At score 4, Catch Me overlaps Login and sits beneath the iframe.

Catch Me!

Score: 4

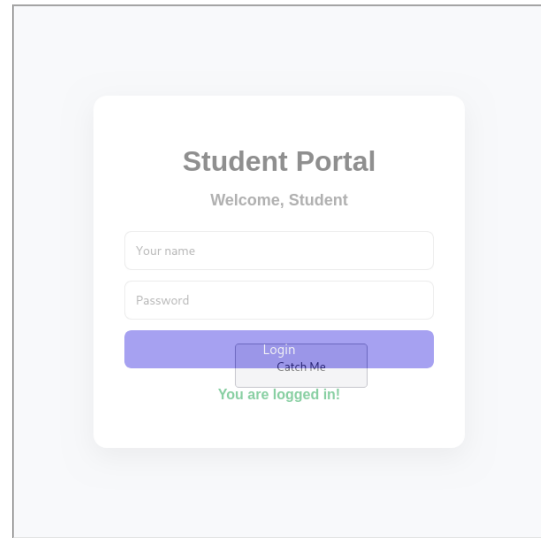


Figure 8: After the next click, the portal displays “You are logged in!; the game score remains 4.

Interaction sequence

1. The user clicks Catch Me four times. The game updates its score and reaches the threshold.
2. The fourth click moves Catch Me to `left: 260px; top: 510px` and lowers its stacking order below the iframe.
3. The next click in the overlapping Login region reaches the embedded portal’s button.
4. The portal executes `login()` and displays its confirmation message. The game’s score stays at 4 because `catchMe()` is not called for that click.

The fourth click prepares the overlap; it is not retroactively delivered to Login. The following click activates Login when it lands within the shared area.

Result and limitations

The supplied final screenshot shows the expected outcome: the portal’s confirmation message appears while the game remains at score 4. The local proof of concept demonstrates an unintended interface action through an overlapping iframe.

The buttons overlap only partially, so a click outside the Login area can miss the target. The fixed coordinates depend on the current page layout. The saved version deliberately retains partial visibility for the demonstrated result; this evidence does not establish a real authentication bypass or a stolen session.

Evidence: `game.html`, `portal.html`, and the eight supplied Firefox screenshots from 15 September 2026. Figures crop unused white margins only; the original screenshots are preserved.

Lab reference: Imane Fouad, UM6P, *TP — Clickjacking: The Button You Didn’t Click*, Sections 1–6, supplied as `TP1_IntroInfoSecu.pdf`.